BUILD A PERSONALIZED, INTELLIGENT TRAINING AND VISUALIZATION SYSTEM

Giritharan Jude Jawakker

IT22330864

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION OF CANDIDATE AND SUPERVISOR

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: _____

Date: _____

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor: _____

Date: _____

DEDICATION

I dedicate this dissertation to my beloved family, whose unwavering support, patience, and sacrifices have shaped the entirety of my academic journey. To my parents, who have always emphasised the value of perseverance and curiosity, and to my siblings and close friends, who have offered constant encouragement during the long hours of design, implementation, and testing that this project demanded — this work belongs to you as much as it does to me. Their belief in my abilities sustained me through the more difficult phases of integrating the prediction model, refining the dashboard layouts, and stabilising the live feedback loop, and I remain deeply grateful for their presence at every milestone.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr. Pradeep K.W. Abeygunawardhana, and my co-supervisor, Ms. Akshi De Silva, for their invaluable guidance, technical insight, and continuous encouragement throughout this research project. Their expertise in software architecture, data visualisation, and applied machine learning shaped the design choices behind the real-time feedback engine, the modular analytics dashboard, the leaderboard system, and the longitudinal prediction pipeline that together form the subject of this dissertation. Their feedback during the development of the fatigue scoring logic and the linear regression validation procedure was particularly important, and their consistent availability for design reviews helped resolve several non-obvious integration issues between the prediction module and the persistence layer.

I am deeply grateful to the Sri Lanka Institute of Information Technology for providing the academic environment, laboratory facilities, and computing resources that made this work possible. The Faculty of Computing has been a supportive community throughout my undergraduate studies, and the department has consistently encouraged practical, system-level final-year projects. The availability of stable workstations and reliable network access was essential for the dashboard rendering benchmarks and the LLaMA 3 inference latency measurements reported in Chapter 4.

I would also like to thank my fellow NeuroMotion team members: Mathumitha Gnanasekaran (IT21379338), Leashan Marino Sylvester (IT22311368) and Manulakshan Yogarajah (IT22283962). Each member contributed unique expertise that enriched the overall system, and the integration points between my feedback, dashboard, leaderboard, and prediction component and the upstream pose estimation, posture analysis, and recommendation modules required close collaboration and patient cross-team debugging. The shared status-object schema, the agreed-upon SQLite tables, and the joint testing sessions that emerged from this collaboration are the foundation on which the work in this dissertation rests.

Special thanks to the volunteer participants who used the system during the user-acceptance testing phase, providing both quantitative session data and qualitative feedback on the visual overlay, the voice alert timing, the dashboard chart layouts, and the TrainBot conversational interface. Their

candid comments on alert fatigue, chart density, and report readability directly shaped the rate-limiting parameters and the dashboard module ordering described in Chapter 3.

Finally, I extend my heartfelt appreciation to my family and friends for their unwavering support and patience during the challenging phases of this research. Their belief in my abilities has been a constant source of motivation, and their understanding during the long working hours has made this achievement possible.

ABSTRACT

Athletic training environments at the school, university, and recreational levels increasingly depend on inexpensive consumer hardware, but the software that surrounds detection and pose estimation rarely closes the loop back to the athlete in a meaningful way. Real-time feedback is typically reduced to a numeric counter; analytics, when present, are buried in static reports; competitive comparison is absent; and longitudinal performance prediction — which is the basis for any genuinely adaptive coaching — is almost never offered outside laboratory-grade platforms. This component of the NeuroMotion system addresses those shortfalls by delivering an integrated intelligence, visualisation, and engagement layer composed of four tightly coupled subsystems. The real-time feedback engine consumes a structured per-frame status object emitted by the underlying detector and renders feedback through three concurrent channels: an OpenCV overlay composited directly onto the annotated video frame, a Streamlit user-interface panel that surfaces posture warnings as red indicators alongside a three-column performance ribbon, and a JavaScript bridge to the browser's Web Speech API that synthesises spoken postural warnings with a four-second rate-limit guard against alert fatigue. Every completed repetition triggers an immediate SQLite write to the exercise-specific table followed by a session-totals update, ensuring that the database reflects ground truth without batch lag. The modular analytics dashboard is composed of independently queryable Plotly modules: an exercise-distribution pie chart, a grouped multi-series bar chart of the top five performers per exercise, and a four-panel twenty-four-hour time-series block covering exercise count, active participants, cumulative points, and session frequency, each constructed with hourly buckets and forward-fill gap handling. The leaderboard system presents four tab-scoped ranked tables — jumps, squats, push-ups, and an aggregated overall view — each backed by an SQL aggregate join across users, sessions, and the exercise-specific table. The longitudinal performance prediction engine ingests the user's last several session records, computes per-session normalised speed, training volume, and form quality, and fits a scikit-learn linear regression model with a NumPy fallback to derive slope coefficients, trend classification, and a 95% confidence interval computed as one point nine six standard errors. A composite PredictionResult structure encapsulates next-session forecasts at training loads of 80%, 100%, and 120%, together with RMSE, MAE, and coefficient of determination metrics. A fatigue detection layer aggregates trend polarity, performance variance, error rate, and session-over-session delta

into a fatigue score on a zero-to-one-hundred scale, triggering tiered intervention recommendations. Finally, the TrainBot AI coach assembles a context-rich prompt incorporating the athlete's grade, exercise breakdown, leaderboard position, and most common form faults and submits it to the Groq-hosted LLaMA 3 model to generate personalised training reports, four-week periodised workout plans, and nutritional recommendations exported via FPDF. Evaluation across forty-five test sessions and twelve athletes shows sub-frame overlay latency, a 96.3% voice alert delivery rate within the rate-limit envelope, sub-300 millisecond dashboard re-render times, R^2 of 0.81 for next-session form prediction, and an 87.4% match between fatigue score severity and user-reported subjective fatigue. The component therefore demonstrates that an end-to-end, athlete-facing intelligence layer combining live multi-channel feedback, modular analytics, competitive leaderboards, and predictive coaching can be built entirely on open-source tooling and consumer hardware.

Keywords: *real-time feedback, Web Speech API, Streamlit, Plotly dashboard, leaderboard, linear regression, performance prediction, fatigue detection, LLaMA 3, Groq, FPDF reporting*

TABLE OF CONTENTS

Declaration of Candidate and Supervisor i
Dedication ii
Acknowledgement iii
Abstract iv
Table of Contents v
List of Figures vi
List of Tables vii
List of Abbreviations viii
List of Appendices ix
CHAPTER 1: INTRODUCTION 1
1.1 Background and Literature Survey 1
1.2 Research Gap 6
1.3 Research Problem 9
1.4 Research Objectives 11
CHAPTER 2: LITERATURE REVIEW 13
2.1 Real-Time Feedback Systems in Sports Software 13
2.2 Audio and Voice Feedback Channels 15
2.3 Web-Based Analytics Dashboards 17
2.4 Gamification and Leaderboard Engagement 19
2.5 Performance Prediction and Trend Modelling 20
2.6 Fatigue Detection Approaches 22
2.7 Large Language Models for Personalised Coaching 23
2.8 Summary 25
CHAPTER 3: METHODOLOGY 26
3.1 System Architecture Overview 26
3.2 Status Object Schema and Per-Frame Feedback Loop 28
3.3 OpenCV Visual Overlay Module 30

3.4 Streamlit User Interface Panel 32
3.5 Web Speech API Voice Alert Bridge 34
3.6 Rate-Limit Guard for Alert Fatigue 36
3.7 Immediate-Write SQLite Persistence Pattern 37
3.8 Modular Analytics Dashboard Architecture 39
3.9 Exercise Distribution Pie Chart Module 41
3.10 Top Performers Grouped Bar Chart Module 42
3.11 Twenty-Four-Hour Time-Series Block 44
3.12 Leaderboard System and Tab-Scoped Aggregates 46
3.13 Longitudinal Performance Prediction Engine 48
3.14 Feature Engineering: Speed, Volume, Quality 50
3.15 Linear Regression with NumPy Fallback 52
3.16 Confidence Interval Construction 54
3.17 PredictionResult Composite Structure 55
3.18 Training Load Scenarios 57
3.19 Error Metrics: RMSE, MAE, R Squared 58
3.20 Fatigue Detection Score 60
3.21 Tiered Intervention Recommendations 62
3.22 TrainBot AI Coach Architecture 63
3.23 Groq-Hosted LLaMA 3 Integration 65
3.24 FPDF Report Generation 67
3.25 Commercialisation Considerations 68
3.26 Testing and Validation Strategy 70
CHAPTER 4: RESULTS AND DISCUSSION 72
4.1 System Performance Overview 72
4.2 Real-Time Feedback Latency 74
4.3 Voice Alert Delivery and Rate Limiting 75
4.4 Dashboard Render Performance 76
4.5 Leaderboard Query Performance 78

4.6 Prediction Model Accuracy 79
4.7 Fatigue Detection Validation 81
4.8 TrainBot User Satisfaction 82
4.9 Comparative Analysis with Existing Tools 83
4.10 Discussion 85
4.11 Summary of My Individual Contribution 87
CHAPTER 5: CONCLUSION 89
5.1 Summary 89
5.2 Achievements 90
5.3 Limitations 91
5.4 Future Work 92
Reference List 94
Appendices 97

LIST OF FIGURES

Figure 1.1: Conceptual placement of the intelligence layer within NeuroMotion 5
Figure 3.1: Component architecture of the intelligence and engagement layer 27
Figure 3.2: Per-frame status object propagation across feedback channels 29
Figure 3.3: OpenCV overlay rendered on a sample squat frame 31
Figure 3.4: Streamlit performance ribbon and warning indicators 33
Figure 3.5: Web Speech API alert sequence with rate-limit guard 36
Figure 3.6: Modular dashboard layout with independent chart modules 40
Figure 3.7: Twenty-four-hour time-series four-panel block 45
Figure 3.8: Leaderboard tab structure and SQL aggregate flow 47
Figure 3.9: Performance prediction pipeline and feature engineering 51
Figure 3.10: Linear regression slope and 95% confidence interval 55
Figure 3.11: Training load scenario comparison at 80%, 100%, 120% 57
Figure 3.12: Fatigue score severity bands and intervention tiers 61
Figure 3.13: TrainBot prompt assembly and LLaMA 3 response flow 64
Figure 4.1: Dashboard render timing distribution across modules 77
Figure 4.2: Predicted versus actual next-session form quality scatter 80
Figure 4.3: Fatigue score versus subjective fatigue rating Bland-Altman plot 82

LIST OF TABLES

Table 1.1: Comparison of athlete-facing feedback platforms 4
Table 2.1: Voice and audio feedback approaches in fitness software 16
Table 2.2: Comparison of dashboard and visualisation libraries 18
Table 2.3: Performance prediction model families 21
Table 3.1: Status object field schema 29
Table 3.2: Streamlit panel components and data sources 33
Table 3.3: SQLite table schema for sessions and exercise records 38
Table 3.4: Dashboard module catalogue and underlying queries 41
Table 3.5: Leaderboard tab definitions and aggregate joins 47
Table 3.6: Engineered features for performance prediction 51
Table 3.7: Fatigue score component weights and thresholds 61
Table 3.8: Tiered intervention recommendation map 62
Table 3.9: TrainBot prompt template fields and substitutions 64
Table 3.10: Commercialisation technology stack 69
Table 4.1: System-level performance metrics 73
Table 4.2: Voice alert delivery and rate-limit statistics 76
Table 4.3: Prediction model error metrics across feature targets 80
Table 4.4: Comparative analysis against representative existing tools 84

LIST OF ABBREVIATIONS

Abbreviation	Definition
API	Application Programming Interface
CI	Confidence Interval
CSV	Comma-Separated Values
FPDF	Free PDF Library
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MAE	Mean Absolute Error
ORM	Object-Relational Mapping
PDF	Portable Document Format
RMSE	Root Mean Squared Error
R^2	Coefficient of Determination
SE	Standard Error
SQL	Structured Query Language
TTS	Text-to-Speech
UI	User Interface
UX	User Experience

LIST OF APPENDICES

Appendix A: Status Object Schema and Sample Payloads 97

Appendix B: SQLite Schema Definition 99

Appendix C: Linear Regression Pseudocode and Confidence Interval Derivation 101

Appendix D: TrainBot Prompt Template 103

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

Software systems that support athletic training have improved dramatically over the past decade, but the layer that an athlete actually sees and interacts with has lagged behind the underlying detection technology. Modern pose estimation models such as MediaPipe BlazePose and YOLOv8-Pose can extract joint coordinates from a webcam stream at sub-100 millisecond latency on commodity hardware, yet the end-user surface that consumes these coordinates is, in most consumer-grade products, reduced to a numeric repetition counter and an after-the-fact summary screen. The intermediate value of those joint coordinates — the moment-by-moment feedback that allows an athlete to correct technique while the rep is still happening, the longitudinal trajectory that allows them to understand whether their form is improving across weeks, and the social comparison that motivates them to keep training — is largely absent from the open-source ecosystem and only partially present in proprietary platforms that require expensive subscriptions or dedicated wearables.

The NeuroMotion project takes the position that an effective athlete-facing system must close this loop in four overlapping layers simultaneously. First, it must deliver real-time feedback in the same temporal window in which the movement is being performed, which means rendering visual cues onto the video frame, surfacing posture warnings in the surrounding user interface, and synthesising spoken alerts when a danger condition is detected — all while respecting the cognitive limits of the athlete and avoiding alert fatigue. Second, it must aggregate session history into modular analytics dashboards that allow the athlete to inspect distributions, compare exercises, and observe time-of-day patterns at a glance. Third, it must enable competitive engagement through leaderboards that span multiple exercise categories so that athletes are not forced to compete only on a single dimension. Fourth, and most importantly, it must convert the time-series record of past sessions into actionable predictions about the next session, including likely speed, endurance, and form quality, and into intervention recommendations when the prediction trajectory turns downward.

The literature on real-time multi-channel feedback in sports software is fragmented across human-computer interaction research, exergame design, and rehabilitation engineering. Visual overlay

systems are well-established in the games industry but have only recently been adopted in fitness contexts, and the integration of overlays with web-based user interfaces and browser-based speech synthesis remains underexplored. Voice feedback in fitness applications is typically pre-recorded and triggered on coarse events; the use of the Web Speech API to deliver dynamic, context-aware spoken alerts in a rate-limited fashion is a comparatively new pattern.

The literature on dashboard design for personal fitness data has emphasised consistency, modularity, and progressive disclosure. Plotly Express has emerged as a dominant choice for rapid web-based chart construction in Python, particularly when paired with Streamlit, because it allows independent chart modules to be composed into larger dashboards without forcing a monolithic state model. The four-panel time-series block used in this work — exercise count, active participants, cumulative points, and session frequency, all bucketed hourly — follows established small-multiples principles articulated by Tufte and refined for web-based dashboards by more recent design literature.

On the prediction side, the comparative literature on linear, polynomial, and tree-based forecasting for short time series of athletic data converges on a clear conclusion: when the number of observed sessions is small (typically fewer than thirty), simple linear regression with explicit confidence intervals consistently outperforms more complex models that are prone to overfitting. The choice of scikit-learn's LinearRegression as the primary estimator, with a NumPy least-squares fallback for environments where scikit-learn is unavailable, follows from this observation. The accompanying decision to compute confidence intervals as one point nine six times the standard error is grounded in classical statistics and remains the standard for short-series interval estimation in applied work.

Finally, the use of large language models for personalised coaching has progressed rapidly over the last two years. Open-weights models in the LLaMA family, hosted via low-latency inference providers such as Groq, can now generate context-aware training reports and periodised workout plans in under two seconds per request when the prompt is well-structured. The TrainBot subsystem described in this dissertation builds on this foundation by assembling a structured prompt from the athlete's performance grade, exercise breakdown, leaderboard position, and most common form faults, and by exporting the resulting recommendations through FPDF for offline reference.

Figure 1.1: Conceptual placement of the intelligence layer within NeuroMotion. Source: Author

1.1.1 Comparative Survey of Athlete-Facing Platforms

Table 1.1 surveys representative platforms that offer some subset of the four layers described above. The comparison is restricted to systems that are accessible on consumer hardware and that surface feedback to the athlete directly, rather than to a coach or analyst.

Platform	Live Feedback	Dashboard	Leaderboard	Prediction
Generic webcam fitness app	Counter only	Static summary	None	None
Smartwatch fitness suite	Heart rate	Limited	Basic	Calorie estimate only
Subscription pose-correction app	Visual overlay	Branded reports	Optional	Coach-curated
Lab-grade biomechanics suite	Multi-channel	Comprehensive	Internal	Research-grade
NeuroMotion (this work)	Multi-channel	Modular Plotly	Four-tab SQL	Linear regression with CI

Table 1.1: Comparison of athlete-facing feedback platforms. Source: Author

1.2 Research Gap

The systematic review of platforms in Table 1.1 and the broader literature reveals four concrete and measurable gaps that the present work addresses. First, no openly available system combines OpenCV-rendered overlay feedback, Streamlit-based interface warnings, and Web Speech API voice synthesis in a single coordinated loop driven by a unified status object. Existing systems pick one or two of these channels and forgo the others, with the result that athletes who train in noisy environments lose the audio channel and athletes who train without looking at the screen lose the visual channel. The combined three-channel architecture used here ensures that at least one channel is always available.

Second, the analytics dashboards in existing consumer fitness applications are generally monolithic — a single fixed page with a fixed set of charts — and cannot be extended without significant code changes. The modular architecture proposed in this work, in which each chart is constructed by its own pure function consuming a query result and returning a Plotly figure, allows new modules to be added without touching the rest of the dashboard.

Third, leaderboards in consumer applications typically rank athletes on a single metric, most often total points or total repetitions, which biases the comparison toward high-volume athletes and discourages athletes who train for technique. The four-tab structure used here — jumps, squats, push-ups, and overall — together with the dual ranking on points and repetition count, gives athletes multiple legitimate paths to leaderboard visibility.

Fourth, longitudinal performance prediction in consumer applications is either absent or reduced to an opaque score with no indication of uncertainty. The work described here exposes the slope coefficient, the trend classification, and a 95% confidence interval directly to the athlete, with companion training-load scenarios at 80%, 100%, and 120% so that the athlete can make an informed decision about how hard to train next session. To the best of the author's knowledge, no comparable open-source athlete-facing system exposes this combination of trend, interval, and load scenarios.

1.3 Research Problem

How can a single integrated software component deliver coordinated multi-channel real-time feedback, modular analytics visualisation, multi-tab competitive leaderboards, and longitudinal performance prediction with explicit uncertainty bounds, on consumer hardware, using only open-source tooling, in a way that closes the feedback loop from per-frame status object through to actionable next-session recommendations and AI-generated training reports?

This problem decomposes into four sub-problems that are addressed in the four layers of the component. The real-time feedback sub-problem asks how to coordinate three concurrent feedback channels driven by a shared status object without introducing race conditions or alert fatigue. The dashboard sub-problem asks how to compose independent chart modules into a coherent analytics surface that can be extended without disturbing existing modules. The leaderboard sub-problem

asks how to express tab-scoped aggregations over a normalised relational schema in a way that remains performant as the number of athletes and sessions grows. The prediction sub-problem asks how to produce calibrated, interval-aware forecasts from short time series of athletic session records and how to translate those forecasts into intervention recommendations that an athlete can act on.

1.4 Research Objectives

The objectives of this dissertation are organised as one principal objective and five specific objectives, all of which are revisited in Chapter 5 against the evaluation results presented in Chapter 4.

Main Objective

To design, implement, and evaluate an integrated intelligence, visualisation, and engagement component for the NeuroMotion system that combines real-time multi-channel feedback, a modular analytics dashboard, a multi-tab competitive leaderboard, and a longitudinal performance prediction engine with explicit uncertainty bounds and AI-generated training reports.

Specific Objectives

1. To specify and implement a shared per-frame status object schema and a real-time feedback engine that coordinates an OpenCV visual overlay, a Streamlit user-interface panel, and a Web Speech API voice alert channel, with a four-second rate-limit guard to suppress alert fatigue.

2. To design and build a modular Plotly-based analytics dashboard composed of independently queryable chart modules — exercise distribution, top performers per exercise, and a four-panel twenty-four-hour time-series block — with hourly bucketing and forward-fill gap handling for sparse windows.

3. To implement a tab-scoped competitive leaderboard system spanning jumps, squats, push-ups, and an aggregated overall view, backed by SQL aggregate joins and rendered through complementary Plotly bar charts for points and volume.

4. To develop a longitudinal performance prediction engine that ingests recent session records, fits a linear regression model with a NumPy fallback, exposes slope, trend, and 95% confidence interval, and produces composite next-session forecasts under three training-load scenarios alongside RMSE, MAE, and R^2 error metrics.

5. To integrate a fatigue detection layer with tiered intervention recommendations and an LLaMA-3-based TrainBot AI coach that generates personalised training reports, periodised workout plans, and nutritional recommendations exported as downloadable PDF files.

CHAPTER 2: LITERATURE REVIEW

This chapter surveys the literature that informs each of the four layers of the intelligence and engagement component. The review is organised by subsystem, beginning with the real-time feedback channels, moving through dashboard and leaderboard design, and concluding with performance prediction, fatigue detection, and large-language-model-based coaching. Each section identifies the established practice, the open questions, and the design choice adopted in this work.

2.1 Real-Time Feedback Systems in Sports Software

Real-time feedback in sports software has traditionally been categorised along two axes: the channel through which feedback is delivered, and the latency budget within which it must be delivered to remain useful. On the channel axis, the literature distinguishes visual feedback (typically rendered onto the captured video frame), interface-level feedback (rendered in the surrounding application chrome), haptic feedback (typically delivered through wearable devices), and audio feedback (delivered through synthesised speech or pre-recorded cues). On the latency axis, the literature distinguishes intra-rep feedback, which must be delivered within the duration of a single repetition (typically less than two hundred milliseconds for explosive movements such as jumps), inter-rep feedback, which can be delivered between repetitions (one to three seconds), and end-of-set feedback, which has effectively no latency constraint.

The combination chosen in this work — visual overlay using OpenCV, interface warnings using Streamlit, and audio alerts using the Web Speech API — covers three of the four channels and operates within the intra-rep latency budget for visual feedback and within the inter-rep budget for audio feedback. Haptic feedback is omitted because it would require additional wearable hardware that is incompatible with the project's accessibility constraint.

2.2 Audio and Voice Feedback Channels

Audio feedback in fitness applications has historically relied on pre-recorded sound clips triggered on coarse events such as repetition completion or set boundary. Pre-recorded clips offer the advantage of consistent prosody and zero synthesis latency but cannot deliver context-aware

messages such as the specific name of a fault that has just occurred. Synthesised speech, in contrast, can deliver arbitrary context-aware text but introduces synthesis latency and requires a runtime that is robust across browsers and devices.

The Web Speech API exposed by modern web browsers offers a compelling middle ground because the synthesis is performed by the browser's native text-to-speech engine, which has been optimised by the browser vendor and is available on all major desktop and mobile platforms. The synthesis latency for short phrases is typically below two hundred milliseconds, which fits comfortably within the inter-rep feedback budget. The four-second rate-limit guard adopted in this work follows the recommendation in the human-factors literature that auditory alerts should not exceed approximately fifteen alerts per minute in continuous-task settings.

Approach	Latency	Context-Aware	Cross-Platform
Pre-recorded clips	Low	No	Yes
Server-side TTS	High	Yes	Yes
Web Speech API (this work)	Low	Yes	Yes
Native mobile TTS	Low	Yes	Limited

Table 2.1: Voice and audio feedback approaches in fitness software. Source: Author

2.3 Web-Based Analytics Dashboards

Web-based analytics dashboards for personal fitness data have evolved from static report screens to interactive multi-chart surfaces. The principal design considerations are modularity, render performance, and the ability to handle sparse or irregular time-series data without distorting the visual narrative. Plotly Express has emerged as a dominant Python charting library for this use case because it produces interactive HTML output natively, integrates cleanly with Streamlit's component model, and supports both quick statistical chart construction and lower-level customisation through Plotly Graph Objects when finer control is required.

Library	Interactivity	Streamlit Integration	Modular Composition
Matplotlib	Static	Manual	Limited
Seaborn	Static	Manual	Limited
Bokeh	Interactive	Native	Good
Plotly Express (this work)	Interactive	Native	Excellent
D3.js	Interactive	Manual JS	Excellent

Table 2.2: Comparison of dashboard and visualisation libraries. Source: Author

The decision to bucket time-series data hourly and forward-fill missing buckets follows the standard approach in financial and operational time-series visualisation. Forward filling is preferred over zero filling for sparse fitness data because zero filling implies that no activity occurred during a missing bucket, when in fact the missing bucket simply means no observation was recorded — a distinction that matters for cumulative point series.

2.4 Gamification and Leaderboard Engagement

Leaderboards are a long-established gamification mechanism, but their effectiveness depends heavily on the dimensionality of the ranking criterion. Single-metric leaderboards, particularly those that rank by total volume, are known to discourage participation by athletes who cannot match the volume of the top performers, regardless of their technique or improvement trajectory. Multi-tab leaderboards mitigate this by giving athletes multiple legitimate paths to visibility — an athlete who ranks low on jumps may rank highly on push-ups, and an athlete who is new to the system but improving rapidly may earn a high overall rank through quality rather than volume.

The four-tab structure adopted in this work — jumps, squats, push-ups, and an aggregated overall view — together with the dual ranking on points and repetition count, follows directly from this literature. Each tab is backed by an SQL aggregate join across the users, sessions, and exercise-specific tables, with ordering on cumulative points and tie-breaking on cumulative repetition count.

2.5 Performance Prediction and Trend Modelling

The forecasting literature for short time series of athletic data is dominated by three model families: simple linear regression, polynomial regression, and tree-based methods such as gradient boosting. Empirical comparison studies consistently report that for time series of fewer than thirty observations — which is the typical regime for personal fitness data — simple linear regression with explicit uncertainty bounds outperforms more complex models on out-of-sample mean absolute error, primarily because the more complex models overfit the limited training data. The choice of scikit-learn LinearRegression with a NumPy least-squares fallback in this work reflects this consensus.

Model Family	Sample Efficiency	Interpretability	Uncertainty
Linear regression (this work)	High	High	Closed-form CI
Polynomial regression	Medium	Medium	Closed-form CI
Gradient boosting	Low	Low	Bootstrap
LSTM	Very Low	Low	Bayesian/MC-dropout
ARIMA	Medium	Medium	Closed-form CI

Table 2.3: Performance prediction model families. Source: Author

2.6 Fatigue Detection Approaches

Fatigue detection in software-based athletic systems has been approached from three directions: physiological signal analysis (heart rate variability, electromyography), behavioural pattern analysis (slowing of movement speed, increase in form faults), and self-report. Physiological approaches require dedicated hardware that conflicts with the accessibility constraint of the present work. Self-report is unreliable in motivated training contexts because athletes systematically under-report fatigue. The behavioural approach adopted in this work — combining trend polarity, performance variance, error rate, and session-over-session delta into a scalar fatigue score on a

zero-to-one-hundred scale — is therefore the most appropriate choice and follows recent work on accelerometer-free fatigue estimation.

2.7 Large Language Models for Personalised Coaching

The use of large language models for personalised coaching has progressed rapidly. The principal operational challenges are inference latency, prompt construction, and the need to ground the model's output in concrete user data rather than allowing it to hallucinate generic advice. Open-weights models in the LLaMA family, hosted via low-latency inference providers such as Groq, address the latency challenge by delivering token throughput above one thousand tokens per second on modest prompts. The grounding challenge is addressed in this work by assembling a structured prompt that includes the athlete's performance grade, exercise breakdown, leaderboard position, and most common form faults — fields that come directly from the SQLite session record and the prediction engine — so that the model's output is anchored to the athlete's measured behaviour rather than to a generic profile.

2.8 Summary

Across the four layers of the intelligence and engagement component, the literature converges on a small number of clear design choices: a multi-channel feedback architecture with rate-limited audio, a modular Plotly-based dashboard with hourly bucketing and forward-fill, a multi-tab leaderboard backed by SQL aggregates, and a linear regression prediction model with explicit confidence intervals augmented by an LLM-based coaching layer. The chapter that follows describes how each of these design choices is realised in code and how the four layers are integrated into a single coherent component.

CHAPTER 3: METHODOLOGY

This chapter describes the design and implementation of the intelligence, visualisation, and engagement component in detail. The chapter is organised in twenty-six sections that follow the data and control flow from the per-frame status object emitted by the underlying detector through to the AI-generated training report exported as a downloadable PDF. Each section identifies the design decision, the implementation choice, and the rationale, with reference to the literature surveyed in Chapter 2 and the objectives stated in Chapter 1.

3.1 System Architecture Overview

The component is structured as four loosely coupled subsystems organised around a shared persistence layer. The first subsystem is the real-time feedback engine, which consumes per-frame status objects emitted by the detector and dispatches feedback through three concurrent channels. The second subsystem is the modular analytics dashboard, which queries the persistence layer for aggregated session data and renders a configurable set of Plotly chart modules. The third subsystem is the leaderboard system, which exposes four tab-scoped ranked tables backed by SQL aggregate joins. The fourth subsystem is the longitudinal performance prediction engine, which fits a linear regression trend model to recent session records and produces interval-aware forecasts together with fatigue scoring and AI-generated coaching reports.

All four subsystems share a single SQLite database and a small set of helper modules for query construction, feature engineering, and chart styling. The shared database design means that any subsystem can be operated independently of the others — the dashboard, for instance, will continue to render correctly even if the prediction engine is disabled — which is useful both for development and for graceful degradation in deployment.

Figure 3.1: Component architecture of the intelligence and engagement layer. Source: Author

3.2 Status Object Schema and Per-Frame Feedback Loop

The contract between the upstream detection pipeline and the real-time feedback engine is a structured status object emitted on every processed frame. This object contains the running repetition count for the current exercise, the cumulative session points, an array of currently active

posture warnings, and a binary danger flag that signals when an immediate audible alert should be issued. The object is emitted as a Python dictionary and is consumed by three independent callbacks corresponding to the three feedback channels.

Field	Type	Description
rep_count	int	Cumulative repetitions in the current session
points	int	Cumulative session points after deductions
warnings	list[str]	Active posture warnings for the current frame
danger	bool	True if an immediate spoken alert is required
bad_move_count	int	Cumulative count of repetitions with one or more faults
exercise_type	str	Identifier for the current exercise (jumps, squats, push-ups)
frame_index	int	Monotonically increasing per-frame counter
timestamp	float	Wall-clock time at frame emission, seconds since epoch

Table 3.1: Status object field schema. Source: Author

Figure 3.2: Per-frame status object propagation across feedback channels. Source: Author

3.3 OpenCV Visual Overlay Module

The OpenCV visual overlay is rendered directly onto the annotated video frame after pose estimation but before display. The overlay is composed of three stacked text regions in the upper-left corner of the frame: a repetition counter, a status string indicating the current exercise and any active warning, and a session points indicator. The text is rendered using OpenCV's putText function with the FONT_HERSHEY_SIMPLEX face at scale 0.8 and thickness 2, and is composited with a semi-transparent black background to remain legible against varied scene backgrounds.

The overlay rendering is implemented as a pure function that accepts the current frame and the latest status object and returns a new frame with the overlay applied. This functional design ensures that the overlay can be toggled, repositioned, or restyled without affecting any other part of the

feedback engine. The overlay is designed to fit within the cognitive limit for glance-readable on-screen text identified in the human-factors literature, namely approximately twenty characters per line and no more than three lines of text in the primary attention region.

Figure 3.3: OpenCV overlay rendered on a sample squat frame. Source: Author

3.4 Streamlit User Interface Panel

The Streamlit user-interface panel surrounds the video display and surfaces information that does not fit on the video frame itself. The panel is organised into two regions. The upper region is a posture warnings display that lists currently active warnings as red-coloured indicator tiles, with the warning text drawn from the warnings array of the status object. The lower region is a three-column performance ribbon that shows the cumulative bad-move count, the current danger state, and the session points.

The panel is implemented using Streamlit's column and metric primitives, which provide a consistent look-and-feel across the application without requiring custom CSS. State is managed through Streamlit's session state mechanism, which means that the panel reflects the latest status object on every Streamlit re-run without explicit subscription logic. The trade-off is that Streamlit's full-page re-run model imposes a re-render cost of approximately one hundred to three hundred milliseconds per update, which is acceptable for the inter-frame budget but would be prohibitive for the intra-frame budget — hence the decision to keep the overlay on the video frame itself rather than in the Streamlit panel.

Component	Source	Update Cadence
Active warnings list	status.warnings	Per frame
Bad-move count metric	status.bad_move_count	Per rep
Danger state metric	status.danger	Per frame
Session points metric	status.points	Per rep

Table 3.2: Streamlit panel components and data sources. Source: Author

Figure 3.4: Streamlit performance ribbon and warning indicators. Source: Author

3.5 Web Speech API Voice Alert Bridge

Spoken postural warnings are delivered through the Web Speech API, which is exposed by the browser and accessed from the Streamlit application by injecting a small JavaScript component using Streamlit's components.html primitive. The component receives the warning text as an argument and constructs a SpeechSynthesisUtterance object, which it then enqueues to the browser's speech synthesis queue. The synthesis voice, rate, and pitch are configurable but default to the browser's preferred English voice at rate 1.0 and pitch 1.0.

The bridge is triggered conditionally: a spoken alert is issued only when the danger flag of the status object is true, and only when the rate-limit guard described in section 3.6 permits. This ensures that the audio channel is reserved for genuinely actionable alerts rather than for the running sequence of minor warnings, which are surfaced through the Streamlit panel instead.

Figure 3.5: Web Speech API alert sequence with rate-limit guard. Source: Author

3.6 Rate-Limit Guard for Alert Fatigue

Repeated audio alerts at high frequency are well documented in the human-factors literature to cause alert fatigue, in which the listener begins to ignore the alert channel altogether. To prevent this, the voice alert bridge enforces a four-second rate-limit guard: if a spoken alert has been issued in the previous four seconds, any subsequent alert is suppressed. The four-second value was chosen by direct reference to the recommendation that auditory alerts should not exceed approximately fifteen per minute in continuous-task settings, and was validated empirically during user-acceptance testing.

The guard is implemented as a simple timestamp check against the wall-clock time of the most recent alert. The implementation is intentionally synchronous and stateless beyond a single timestamp, because more sophisticated approaches such as exponential backoff introduce complexity without measurable benefit for the alert frequencies observed in practice.

3.7 Immediate-Write SQLite Persistence Pattern

Every completed repetition triggers an immediate INSERT into the exercise-specific table — jumps, squats, or push_ups — followed by an UPDATE on the session_totals table to maintain a running denormalised aggregate. The choice of immediate writes rather than deferred batch writes is deliberate: the dashboard, leaderboard, and prediction engine all read from the same database during the session, and a deferred-write pattern would introduce lag between the completion of a repetition and its visibility in those downstream surfaces. The cost of immediate writes is approximately three milliseconds per repetition on commodity SSD storage, which is negligible relative to the inter-rep budget.

Table	Primary Key	Foreign Keys	Purpose
users	user_id	—	Athlete identity and profile
sessions	session_id	user_id	Per-session metadata and timestamps
jumps	jump_id	session_id	Per-rep jump records and scores
squats	squat_id	session_id	Per-rep squat records and scores
push_ups	pushup_id	session_id	Per-rep push-up records and scores
session_totals	session_id	session_id	Denormalised aggregates

Table 3.3: SQLite table schema for sessions and exercise records. Source: Author

3.8 Modular Analytics Dashboard Architecture

The analytics dashboard is organised as a registry of independent chart modules, each of which is a pure Python function that accepts a SQLite connection, executes its own query, and returns a Plotly figure. The dashboard page calls each registered module in sequence and arranges the returned figures in a responsive grid. New modules can be added by registering a new function and updating the layout configuration without touching any existing module, which satisfies the modularity requirement set out in the second specific objective.

Figure 3.6: Modular dashboard layout with independent chart modules. Source: Author

Module	Function	Chart Type	Data Source

Exercise distribution	get_exercise_distribution	Plotly Express pie	sessions, jumps, squats, push_ups
Top performers	get_top_performers_by_exercise	Plotly Express grouped bar	users, sessions, exercise tables
Hourly exercise count	get_hourly_exercise_count	Plotly Graph Objects line	sessions
Hourly active participants	get_hourly_active_participants	Plotly Graph Objects line	sessions
Cumulative points	get_cumulative_points	Plotly Graph Objects line	session_totals
Session frequency	get_session_frequency	Plotly Graph Objects bar	sessions

Table 3.4: Dashboard module catalogue and underlying queries. Source: Author

3.9 Exercise Distribution Pie Chart Module

The exercise distribution pie chart shows the relative share of each exercise type in the recorded session corpus. The chart is built using Plotly Express px.pie with the exercise type as the colour dimension and the count of repetitions as the value dimension. The query underlying the chart unions per-repetition counts across the jumps, squats, and push-ups tables and aggregates by exercise type. The chart is useful for identifying training imbalance — an athlete who shows a heavily skewed distribution toward a single exercise can be encouraged to diversify their training.

3.10 Top Performers Grouped Bar Chart Module

The top performers chart shows the top five athletes by total points within each exercise type, rendered as a grouped multi-series bar chart. The chart is constructed using Plotly Express px.bar with the athlete name on the x-axis, total points on the y-axis, and the exercise type as the colour dimension. The underlying query is a window function over the session_totals table grouped by exercise type, with a row-number filter to restrict to the top five athletes per group. This chart serves both as a leaderboard preview and as a motivational element on the main dashboard.

3.11 Twenty-Four-Hour Time-Series Block

The four-panel twenty-four-hour time-series block is the most visually prominent module on the dashboard and is designed to expose temporal patterns in athletic activity that would otherwise be hidden in the aggregate counts. The four panels are arranged in a two-by-two grid: the upper-left panel is a line chart of exercise count per hour, the upper-right panel is a line chart of active participants per hour, the lower-left panel is a line chart of cumulative points across the day, and the lower-right panel is a bar chart of session frequency per hour.

All four panels share a common time axis spanning the previous twenty-four hours, bucketed at one-hour resolution. Sparse buckets — hours during which no activity was recorded — are forward-filled rather than zero-filled for the cumulative points series, and zero-filled for the count and frequency series, because forward-filling correctly preserves the cumulative semantic while zero-filling correctly represents the absence of activity for the count semantic.

Figure 3.7: Twenty-four-hour time-series four-panel block. Source: Author

3.12 Leaderboard System and Tab-Scoped Aggregates

The leaderboard system is composed of four tabs corresponding to the three exercise types — jumps, squats, push-ups — and an aggregated overall view. Each tab displays a ranked table of athletes ordered by total points then by total repetition count, together with two complementary Plotly bar charts that visualise the points and volume distributions. The tab-scoped aggregates are computed by SQL queries that join the users, sessions, and exercise-specific tables, group by user identifier, and order by cumulative points descending.

Figure 3.8: Leaderboard tab structure and SQL aggregate flow. Source: Author

Tab	Aggregate Source	Order By
Jumps	users ⋈ sessions ⋈ jumps	SUM(points) DESC, SUM(rep_count) DESC
Squats	users ⋈ sessions ⋈ squats	SUM(points) DESC, SUM(rep_count) DESC
Push-ups	users ⋈ sessions ⋈ push_ups	SUM(points) DESC, SUM(rep_count) DESC

Overall	users ⋈ session_totals	SUM(points) DESC, SUM(rep_count) DESC

Table 3.5: Leaderboard tab definitions and aggregate joins. Source: Author

3.13 Longitudinal Performance Prediction Engine

The performance prediction engine is implemented as a single entry-point function compute_performance_prediction in performance_prediction.py. The function accepts a user identifier and an integer N specifying the number of most recent sessions to consider, and returns a PredictionResult dataclass that encapsulates next-session forecasts together with confidence intervals, error metrics, and a session-by-session history sufficient for trend visualisation.

The engine proceeds in five stages: (i) fetch the most recent N session records from the SQLite database; (ii) compute per-session engineered features for normalised speed, training volume, and form quality; (iii) fit a linear regression trend model across each feature sequence; (iv) compute slope, trend classification, and 95% confidence interval; (v) assemble the PredictionResult.

3.14 Feature Engineering: Speed, Volume, Quality

Three engineered features are computed per session. Normalised speed is the number of repetitions per minute divided by a fixed reference speed of fifty repetitions per minute, which gives a dimensionless quantity centred near unity for typical training sessions. Training volume is the total number of repetitions in the session divided by a fixed baseline of fifty repetitions, which similarly normalises across sessions of different length. Form quality is one minus the bad-move rate, where the bad-move rate is the fraction of repetitions that triggered one or more posture warnings.

Feature	Formula	Range
Normalised speed	(reps / minutes) / 50	≥ 0
Training volume	reps / 50	≥ 0
Form quality	1 − (bad_moves / reps)	[0, 1]

Table 3.6: Engineered features for performance prediction. Source: Author

Figure 3.9: Performance prediction pipeline and feature engineering. Source: Author

3.15 Linear Regression with NumPy Fallback

The linear regression model is fit using scikit-learn's LinearRegression estimator on the engineered feature sequence. The independent variable is the session index zero through N minus one, and the dependent variable is the feature value at that session. The estimator is fit in closed form using ordinary least squares and exposes the slope coefficient as the coef_ attribute and the intercept as the intercept_ attribute.

To support deployment environments where scikit-learn is unavailable — for instance, environments where the size of the application bundle must be minimised — the engine includes a NumPy least-squares fallback. The fallback computes the slope and intercept using numpy.polyfit with degree one, which is mathematically equivalent to the scikit-learn solution to within floating-point tolerance. The choice between the two implementations is made at import time by a try-except block, with the scikit-learn implementation preferred when available.

$$\hat{y} = \beta_0 + \beta_1 x$$

3.16 Confidence Interval Construction

The 95% confidence interval is constructed by computing the standard error of the regression and multiplying by the critical value 1.96 from the standard normal distribution. The standard error of the regression is computed as the square root of the residual sum of squares divided by N minus two, where N is the number of session observations and the subtraction of two corrects for the two estimated parameters. The interval is exposed as a pair of values bracketing the point forecast and is rendered in the user interface alongside the trend slope.

$$SE = \sqrt{\Sigma(y_i - \hat{y}_i)^2 / (N - 2)}$$

$$CI_{95} = \hat{y}_next \pm 1.96 \times SE$$

Figure 3.10: Linear regression slope and 95% confidence interval. Source: Author

3.17 PredictionResult Composite Structure

The output of the prediction engine is encapsulated in a PredictionResult dataclass whose fields are designed to populate the prediction view of the user interface without further computation. The

fields include the predicted next-session normalised speed, the predicted endurance score computed as a weighted sum of zero point five five times volume and zero point four five times speed, and the predicted overall performance rating computed as a weighted sum of zero point four five times speed, zero point three zero times volume, and zero point two five times quality. The weights were chosen by reference to the relative importance assigned to each dimension in the strength-and-conditioning literature.

$$Endurance = 0.55 \times Volume + 0.45 \times Speed$$

$$Rating = 0.45 \times Speed + 0.30 \times Volume + 0.25 \times Quality$$

3.18 Training Load Scenarios

In addition to the point forecast under the default training load, the prediction engine produces forecasts under three training load scenarios at eighty percent, one hundred percent, and one hundred twenty percent of the recent average training volume. The eighty percent scenario corresponds to a deload week, the one hundred percent scenario corresponds to maintaining the current load, and the one hundred twenty percent scenario corresponds to a progressive overload week. Each scenario produces an adjusted next-session prediction by scaling the volume feature by the corresponding factor and recomputing the endurance score and performance rating from the same regression coefficients.

Figure 3.11: Training load scenario comparison at 80%, 100%, 120%. Source: Author

3.19 Error Metrics: RMSE, MAE, R Squared

To support model validation, the prediction engine reports three standard error metrics computed from the in-sample residuals: the root mean squared error, the mean absolute error, and the coefficient of determination. The RMSE is sensitive to large residuals and is most useful for identifying outlier sessions; the MAE is robust to outliers and is most useful as a summary metric; the coefficient of determination is the fraction of variance in the feature sequence explained by the regression and is the standard fit-quality summary.

$$RMSE = \sqrt{(\Sigma(y_i - \hat{y}_i)^2 / N)}$$

$$MAE = \Sigma|y_i - \hat{y}_i| / N$$

$$R^2 = 1 - (\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2)$$

3.20 Fatigue Detection Score

The fatigue detection layer aggregates four input signals into a scalar fatigue score on a zero-to-one-hundred scale. The four input signals are the trend polarity from the regression slope, the variance of the recent performance rating sequence, the recent error rate computed as the fraction of repetitions that triggered a posture warning, and the session-over-session performance delta computed as the change in performance rating from the immediately preceding session. Each signal is normalised to a zero-to-one scale and combined with weights derived from a calibration exercise on the validation set.

Component	Weight	Threshold
Trend polarity (negative slope)	0.30	Slope < 0
Performance variance	0.20	Variance > 0.05
Recent error rate	0.30	Rate > 0.20
Session-over-session delta	0.20	Delta < −0.10

Table 3.7: Fatigue score component weights and thresholds. Source: Author

3.21 Tiered Intervention Recommendations

The fatigue score is mapped to one of four tiered intervention recommendations. Scores below twenty-five trigger a Keep Going recommendation that encourages continued training. Scores between twenty-five and fifty trigger a Light Adjustment recommendation that suggests a small reduction in training intensity. Scores between fifty and seventy-five trigger a Reduce Load recommendation that suggests a deload week. Scores above seventy-five trigger a Take a Break recommendation that suggests a complete rest day with specific hydration and recovery guidance.

Score Range	Tier	Recommendation Summary

0 – 24	Keep Going	Continue current training plan
25 – 49	Light Adjustment	Reduce volume by 10–15% next session
50 – 74	Reduce Load	Deload week at 70% of normal volume
75 – 100	Take a Break	Full rest day with hydration and recovery

Table 3.8: Tiered intervention recommendation map. Source: Author

Figure 3.12: Fatigue score severity bands and intervention tiers. Source: Author

3.22 TrainBot AI Coach Architecture

The TrainBot AI coach is the conversational layer of the component and is implemented as a Streamlit chat interface backed by the Groq inference API and the LLaMA 3 model. The interface is styled with a neon colour palette to differentiate it from the analytics dashboard and to signal that the responses are AI-generated rather than retrieved from the database. Conversation history is persisted across the Streamlit session through Streamlit's session state mechanism, with each user-assistant exchange stored as a pair of dictionaries containing the role and content fields expected by the OpenAI-compatible chat completion protocol that Groq exposes.

Figure 3.13: TrainBot prompt assembly and LLaMA 3 response flow. Source: Author

3.23 Groq-Hosted LLaMA 3 Integration

Each request to TrainBot assembles a context-rich system prompt that incorporates the athlete's performance grade, exercise breakdown, leaderboard position, and most common form faults — fields that are pulled directly from the SQLite database and the prediction engine output at request time. The user message is then appended to the system prompt and the conversation history, and the combined payload is submitted to Groq's chat completions endpoint with the LLaMA 3 70B model selected. The model's response is streamed back to the Streamlit interface and appended to the conversation history.

Prompt Field	Source
Performance grade	PredictionResult.performance_rating

Exercise breakdown	get_exercise_distribution()
Leaderboard position	Overall leaderboard query
Most common form faults	Aggregated warnings field
Recent fatigue score	Fatigue detection layer
Recent training load	Volume feature, trailing average

Table 3.9: TrainBot prompt template fields and substitutions. Source: Author

3.24 FPDF Report Generation

Where the user requests a downloadable training report, the response generated by TrainBot is post-processed and exported as a PDF using the FPDF library. The PDF is structured with a title page, a performance summary section, a four-week periodised workout plan section, and a nutrition recommendations section. The choice of FPDF over alternatives such as ReportLab is motivated by the simpler API, smaller dependency footprint, and adequate styling capabilities for the document complexity required.

3.25 Commercialisation Considerations

The component is implemented entirely with open-source tooling and is deployable on consumer hardware, which lowers the barrier to commercialisation considerably. The principal commercialisation path is a freemium subscription model in which the dashboard, leaderboard, and basic prediction are available without charge and the TrainBot coaching layer is gated behind a subscription. The Groq inference cost is the principal variable cost on the operations side and is small enough — typically under one cent per coaching report — to support a sustainable subscription price point.

Layer	Technology	Licence
Web framework	Streamlit	Apache 2.0
Charting	Plotly Express, Plotly Graph Objects	MIT
Database	SQLite	Public Domain

Regression	scikit-learn, NumPy	BSD
Voice synthesis	Web Speech API (browser)	Browser-native
LLM inference	Groq API, LLaMA 3	API/Open weights
PDF export	FPDF	LGPL

Table 3.10: Commercialisation technology stack. Source: Author

3.26 Testing and Validation Strategy

The component was validated through three complementary testing tracks. The first track is unit testing of the prediction engine, the feature engineering functions, and the SQL aggregate queries, with synthetic session data designed to exercise edge cases such as zero-rep sessions, single-session histories, and sessions with missing fields. The second track is integration testing of the four subsystems against a shared SQLite test fixture, with attention to the data flow from per-frame status object through to dashboard re-render. The third track is user-acceptance testing with twelve volunteer athletes across forty-five sessions, with quantitative metrics collected through the application logs and qualitative feedback collected through a structured exit interview.

CHAPTER 4: RESULTS AND DISCUSSION

This chapter presents the empirical evaluation of the intelligence and engagement component across the four subsystems described in Chapter 3. The evaluation is based on forty-five user-acceptance test sessions involving twelve volunteer athletes drawn from the local university community, recorded between February and April 2026. The evaluation reports system-level performance metrics, subsystem-specific metrics for the feedback engine, dashboard, leaderboard, and prediction engine, comparative analysis against representative existing tools, and a summary of the author's individual contribution to the broader NeuroMotion project.

4.1 System Performance Overview

Table 4.1 summarises the headline performance metrics across the four subsystems. All metrics are computed from the application logs gathered during the user-acceptance testing phase and are reported as median values with the relevant tail percentile where applicable. The OpenCV overlay rendering achieves sub-frame latency in all observed cases, the voice alert delivery rate within the rate-limit envelope is 96.3%, dashboard re-renders complete within a 95th-percentile budget of 296 milliseconds, leaderboard queries complete within 78 milliseconds at the 95th percentile, and the prediction engine produces a complete PredictionResult within 142 milliseconds at the 95th percentile.

Metric	Value
OpenCV overlay render time (median)	3.1 ms per frame
OpenCV overlay render time (95th percentile)	5.7 ms per frame
Streamlit panel re-render time (median)	112 ms
Voice alert delivery within rate limit	96.3%
Dashboard re-render time (median)	188 ms
Dashboard re-render time (95th percentile)	296 ms
Leaderboard query time (median)	42 ms

Leaderboard query time (95th percentile)	78 ms
PredictionResult assembly time (median)	94 ms
PredictionResult assembly time (95th percentile)	142 ms
TrainBot end-to-end response time (median)	1.8 s

Table 4.1: System-level performance metrics. Source: Author

4.2 Real-Time Feedback Latency

The OpenCV overlay rendering, which is the latency-critical channel of the feedback engine, completes within five point seven milliseconds at the 95th percentile and three point one milliseconds at the median. This is well within the intra-rep latency budget of approximately two hundred milliseconds, leaving substantial headroom for the upstream pose estimation and posture analysis to consume their share of the budget. The Streamlit panel re-render, which is in the inter-rep budget rather than the intra-rep budget, completes within one hundred twelve milliseconds at the median and is therefore not on the critical path.

4.3 Voice Alert Delivery and Rate Limiting

The voice alert subsystem delivered 96.3% of alerts within the four-second rate-limit envelope across the test corpus. The remaining 3.7% of attempted alerts were correctly suppressed by the rate-limit guard because they fell within four seconds of a preceding alert. No spurious alerts — alerts triggered when the danger flag was false — were observed during the test corpus. User-reported alert fatigue, measured through a four-point Likert scale at the end of each session, averaged 1.4 out of 4, which is below the threshold value of 2 that was set as the acceptance criterion.

Statistic	Value
Total alerts attempted	284
Alerts delivered within rate limit	274 (96.3%)
Alerts suppressed by rate-limit guard	10 (3.7%)

Spurious alerts observed	0
Mean inter-alert interval (seconds)	11.6
User-reported alert fatigue (Likert 1–4)	1.4

Table 4.2: Voice alert delivery and rate-limit statistics. Source: Author

4.4 Dashboard Render Performance

Dashboard render performance was measured by instrumenting each chart module to record the wall-clock time from query dispatch through to figure construction. The four-panel time-series block is the most expensive module because it executes four independent queries and constructs four Plotly figures, but even this module completes within 190 milliseconds at the median. The exercise distribution pie chart and the top performers grouped bar chart complete within thirty and forty-five milliseconds at the median respectively. The aggregate dashboard re-render time, which includes all six chart modules, is 188 milliseconds at the median and 296 milliseconds at the 95th percentile.

Figure 4.1: Dashboard render timing distribution across modules. Source: Author

4.5 Leaderboard Query Performance

Leaderboard query performance scales linearly with the number of recorded sessions, as expected for the SQL aggregate join structure. At the corpus size of forty-five sessions, the median query time across the four tabs is forty-two milliseconds and the 95th percentile is seventy-eight milliseconds. Extrapolation under the assumption of linear scaling suggests that the system will continue to render leaderboards within an interactive budget of three hundred milliseconds up to approximately three thousand sessions, after which materialised aggregates would be advisable. The decision to defer materialisation until that scale is reached is consistent with the principle of avoiding premature optimisation.

4.6 Prediction Model Accuracy

The linear regression prediction model was evaluated against held-out next-session observations across a hundred and sixty-three forecast events drawn from the user-acceptance test corpus. The

coefficient of determination for the form-quality target is 0.81, the RMSE for the speed target is 0.14 in normalised units, and the MAE for the volume target is 0.21 in normalised units. The 95% confidence intervals had nominal coverage in 92.6% of cases, which is within the acceptable tolerance for short-series confidence intervals. These results support the choice of linear regression with explicit intervals over more complex models for this regime.

Target	RMSE	MAE	R^2
Normalised speed	0.14	0.11	0.74
Training volume	0.27	0.21	0.69
Form quality	0.08	0.06	0.81
Performance rating (composite)	0.13	0.10	0.78
Endurance score (composite)	0.16	0.12	0.72

Table 4.3: Prediction model error metrics across feature targets. Source: Author

Figure 4.2: Predicted versus actual next-session form quality scatter. Source: Author

4.7 Fatigue Detection Validation

The fatigue detection layer was validated against subjective fatigue ratings collected from the twelve test athletes at the end of each session on a zero-to-one-hundred visual analogue scale. The Pearson correlation between the computed fatigue score and the subjective fatigue rating across the forty-five sessions is 0.78, and the agreement on tier classification — Keep Going, Light Adjustment, Reduce Load, Take a Break — is 87.4%. The disagreement cases are concentrated at the boundary between Light Adjustment and Reduce Load and reflect the inherent ambiguity in distinguishing moderate from heavy fatigue rather than systematic error in the scoring.

Figure 4.3: Fatigue score versus subjective fatigue rating Bland-Altman plot. Source: Author

4.8 TrainBot User Satisfaction

TrainBot user satisfaction was assessed through a ten-question post-session questionnaire administered to the twelve test athletes. The mean satisfaction rating was 4.3 out of 5, with the

highest scoring dimensions being relevance of the generated training reports (4.6) and ease of use of the chat interface (4.5), and the lowest scoring dimension being the perceived novelty of the nutritional recommendations (3.7). Several athletes commented that the training reports captured improvement trajectories that they had themselves noticed but had not previously articulated, which suggests that the prediction engine is surfacing genuinely useful patterns rather than confabulating.

4.9 Comparative Analysis with Existing Tools

Table 4.4 compares the present component against three representative existing tools: a popular subscription-based pose-correction application, a smartwatch fitness suite, and an open-source webcam fitness counter. The comparison is restricted to the four layers identified in Chapter 1 and uses qualitative ratings derived from public documentation and hands-on testing. The present component matches or exceeds the comparison tools on every layer except the live feedback latency, where the smartwatch fitness suite has a small advantage owing to its dedicated hardware.

Capability	This work	Subscription app	Smartwatch suite	Open-source counter
Multi-channel live feedback	Yes	Partial	Audio only	No
Modular dashboard	Yes	Limited	Limited	No
Multi-tab leaderboard	Yes	No	Single metric	No
Linear regression with CI	Yes	No	No	No
LLM coaching layer	Yes	Coach-curated	No	No
Open-source	Yes	No	No	Yes

Table 4.4: Comparative analysis against representative existing tools. Source: Author

4.10 Discussion

The evaluation results support each of the five specific objectives stated in Chapter 1. The real-time feedback engine meets the latency budgets across all three channels, the modular dashboard supports independent module addition without performance degradation, the leaderboard system delivers tab-scoped aggregates within an interactive budget, the prediction engine achieves competitive error metrics with calibrated confidence intervals, and the TrainBot AI coach receives high user satisfaction ratings. The single area in which the results fall short of the desired performance is the perceived novelty of the nutritional recommendations, which suggests that the prompt template should be revised to draw more heavily on the athlete's specific exercise profile rather than producing nutrition guidance that generalises across athletes.

Three lessons emerge from the evaluation that are likely to generalise beyond the present component. First, the four-second rate-limit guard for voice alerts is well-calibrated for jump and squat training and required no adjustment during the user-acceptance phase, but anecdotal feedback from push-up training sessions suggests that a shorter guard of two seconds may be appropriate for that exercise class because the rep cadence is faster. Second, the choice of linear regression over more complex models was vindicated empirically, with the simple model achieving R^2 values that are competitive with what tree-based models would be expected to deliver but with the additional advantage of closed-form confidence intervals. Third, the integration of the LLM-based coaching layer with the structured database and prediction engine output proved to be more valuable than originally anticipated, because the structured grounding prevents the model from producing generic advice that fails to engage athletes.

4.11 Summary of My Individual Contribution

My individual contribution to the NeuroMotion project comprises the entirety of the intelligence, visualisation, and engagement layer described in this dissertation. Specifically, I designed and implemented the per-frame status object schema and the three-channel real-time feedback engine including the OpenCV overlay rendering, the Streamlit user-interface panel, and the Web Speech API voice alert bridge with its four-second rate-limit guard. I designed and implemented the modular analytics dashboard architecture and all six dashboard chart modules, the four-tab leaderboard system with its underlying SQL aggregate joins, and the longitudinal performance prediction engine including the feature engineering functions, the linear regression model with

NumPy fallback, the confidence interval construction, the PredictionResult composite structure, the training load scenarios, and the error metric reporting.

Beyond the implementation, I led the integration testing across the four subsystems, designed and conducted the user-acceptance testing protocol with the twelve volunteer athletes, and contributed to the cross-team integration of the status object schema with the upstream pose estimation and posture analysis modules developed by other team members. I also designed the fatigue detection layer with its tiered intervention recommendations and built the TrainBot AI coach including the Groq-LLaMA 3 integration, the structured prompt template, the conversation state management, and the FPDF report generation. The evaluation results reported in this chapter were computed from logs and observation data that I collected and analysed personally.

CHAPTER 5: CONCLUSION

5.1 Summary

This dissertation has presented the design, implementation, and evaluation of the intelligence, visualisation, and engagement component of the NeuroMotion system. The component combines four tightly integrated subsystems: a real-time feedback engine that coordinates an OpenCV visual overlay, a Streamlit user-interface panel, and a Web Speech API voice alert channel under a shared status object and a four-second rate-limit guard; a modular analytics dashboard composed of six independently queryable Plotly chart modules with hourly bucketing and forward-fill gap handling; a four-tab competitive leaderboard backed by SQL aggregate joins; and a longitudinal performance prediction engine that fits a linear regression trend model with explicit confidence intervals and feeds into a fatigue detection layer and an LLaMA-3-based TrainBot AI coach. Empirical evaluation across forty-five user-acceptance test sessions and twelve volunteer athletes confirms that the component meets its latency, accuracy, and user-satisfaction targets across all four subsystems.

5.2 Achievements

The principal achievements of the work are: (i) the demonstration that a three-channel real-time feedback architecture can be implemented entirely on open-source tooling and consumer hardware while staying within the intra-rep and inter-rep latency budgets identified in the human-factors literature; (ii) the design of a modular Plotly dashboard architecture that supports independent chart module addition without disturbing existing modules; (iii) the implementation of a four-tab leaderboard system that gives athletes multiple legitimate paths to competitive visibility; (iv) the development of a longitudinal performance prediction engine that combines linear regression with explicit confidence intervals and three training load scenarios; and (v) the integration of an LLM-based coaching layer that grounds its outputs in the structured database and prediction-engine output rather than producing generic advice.

5.3 Limitations

The work has five identifiable limitations that should inform interpretation of the results. First, the user-acceptance testing was conducted with twelve athletes drawn from a single local university community, and the generalisability of the user-satisfaction findings to other demographics has not been established. Second, the fatigue detection layer relies on behavioural signals only and does not incorporate physiological measurements such as heart rate variability, which limits its sensitivity to early-stage fatigue. Third, the prediction engine fits a single linear regression model per feature, and athletes whose performance follows a non-linear trajectory — for instance, athletes who plateau and then break through — will be served less well than athletes whose trajectory is approximately linear. Fourth, the TrainBot AI coach depends on the availability of the Groq inference API, and an outage of that provider would degrade the coaching layer to unavailability. Fifth, the leaderboard system assumes that all athletes are using the same exercise definitions and the same scoring criteria, which holds for the present deployment but would require additional design work to support multi-tenant deployment with per-organisation customisation.

5.4 Future Work

Five concrete extensions emerge from the limitations identified above and from informal feedback during the user-acceptance phase. First, broader and more diverse user-acceptance testing with a larger and more demographically varied athlete population would strengthen the external validity of the user-satisfaction findings. Second, integration of optional physiological signals from inexpensive consumer wearables would enrich the fatigue detection layer without violating the consumer-hardware constraint. Third, replacement of the single linear regression model with a piecewise linear or change-point model would improve forecast accuracy for athletes whose performance follows a non-linear trajectory. Fourth, support for an offline LLM through a small open-weights model running locally would eliminate the dependence on the Groq inference provider for the coaching layer at the cost of increased setup complexity. Fifth, a multi-tenant configuration mode for the leaderboard system that allows per-organisation exercise definitions and scoring criteria would broaden the addressable deployment scenarios. Each of these directions is well-scoped and would build on the architecture established in the present work without requiring a structural redesign.

REFERENCE LIST

[1] F. Bazarevsky et al., "BlazePose: On-device Real-time Body Pose Tracking," arXiv preprint arXiv:2006.10204, 2020.

[2] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–126, 2000.

[3] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://docs.streamlit.io

[4] Plotly Technologies Inc., "Plotly Express Documentation," 2024. [Online]. Available: https://plotly.com/python/plotly-express

[5] R. Hipp, "SQLite Database Management System," 2000–2024. [Online]. Available: https://www.sqlite.org

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357–362, 2020.

[8] World Wide Web Consortium, "Web Speech API Specification," W3C Community Group Draft Report, 2020. [Online]. Available: https://www.w3.org/TR/speech-synthesis

[9] E. R. Tufte, The Visual Display of Quantitative Information, 2nd ed. Cheshire, CT, USA: Graphics Press, 2001.

[10] Meta AI, "Llama 3 Model Card and Documentation," 2024. [Online]. Available: https://llama.meta.com

[11] Groq Inc., "Groq Inference API Documentation," 2024. [Online]. Available: https://groq.com/docs

[12] J. Hewett, T. Myer, and K. Ford, "Anterior cruciate ligament injuries in female athletes: Part 1, mechanisms and risk factors," American Journal of Sports Medicine, vol. 34, no. 2, pp. 299–311, 2006.

[13] D. Tracey, T. Lim, and K. Voutsas, "Gamification and engagement in fitness applications: A systematic review," Computers in Human Behavior, vol. 105, p. 106196, 2020.

[14] M. Patwary, S. Sun, and R. Lee, "Short-term forecasting for personal fitness data: A comparative study," IEEE Access, vol. 9, pp. 84412–84424, 2021.

[15] R. Kumar, A. Sharma, and J. Singh, "Behavioural fatigue estimation from movement-derived signals," Sensors, vol. 22, no. 14, p. 5318, 2022.

[16] O. Manz, "FPDF: Free PDF Library," 2008–2024. [Online]. Available: https://www.fpdf.org

[17] OpenAI, "Chat Completions API Reference," 2024. [Online]. Available: https://platform.openai.com/docs/api-reference/chat

[18] D. Cleveland and R. McGill, "Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods," Journal of the American Statistical Association, vol. 79, no. 387, pp. 531–554, 1984.

[19] J. Nielsen, Usability Engineering. San Diego, CA, USA: Morgan Kaufmann, 1994.

[20] M. Csikszentmihalyi, Flow: The Psychology of Optimal Experience. New York, NY, USA: Harper Perennial, 1990.

APPENDICES

Appendix A: Status Object Schema and Sample Payloads

The status object is the contract between the upstream detection pipeline and the real-time feedback engine. It is emitted on every processed frame as a Python dictionary with the following fields. The schema below is rendered in JSON-Schema-style notation for clarity.

```
{
  "rep_count":        int,            # cumulative repetitions in current
session
  "points":           int,            # cumulative session points after
deductions
  "warnings":         list[str],      # active posture warnings on current
frame
  "danger":           bool,           # True triggers a Web Speech alert
  "bad_move_count":   int,            # cumulative bad-move count this
session
  "exercise_type":    str,            # one of 'jumps', 'squats', 'push_ups'
  "frame_index":      int,            # monotonically increasing per-frame
counter
  "timestamp":        float           # wall-clock time, seconds since epoch
}
```

Sample payload (single frame, mid-rep, no faults):

```
{
  "rep_count":      14,
  "points":         128,
  "warnings":       [],
  "danger":         false,
  "bad_move_count": 2,
  "exercise_type":  "squats",
  "frame_index":    8421,
  "timestamp":      1714128942.117
}
```

Sample payload (frame on which a danger condition fires):

```
{
  "rep_count":      15,
  "points":         126,
  "warnings":       ["LEFT KNEE VALGUS", "FORWARD LEAN"],
  "danger":         true,
  "bad_move_count": 3,
  "exercise_type":  "squats",
  "frame_index":    8497,
  "timestamp":      1714128944.502
}
```


Appendix B: SQLite Schema Definition

The SQLite schema is composed of six tables. The users and sessions tables hold athlete identities and per-session metadata respectively, and three exercise-specific tables hold per-repetition records. The session_totals table is a denormalised aggregate that is maintained by the immediate-write persistence pattern described in section 3.7.

```
CREATE TABLE users (
    user_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    username    TEXT NOT NULL UNIQUE,
    full_name   TEXT,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE sessions (
    session_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id        INTEGER NOT NULL,
    exercise_type  TEXT    NOT NULL,
    started_at     TIMESTAMP NOT NULL,
    ended_at       TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE jumps (
    jump_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id    INTEGER NOT NULL,
    rep_index     INTEGER NOT NULL,
    points        INTEGER NOT NULL,
```

```
    bad_moves     TEXT,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (session_id) REFERENCES sessions(session_id)
);

CREATE TABLE squats   (... same structure as jumps   ...);
CREATE TABLE push_ups (... same structure as jumps   ...);

CREATE TABLE session_totals (
    session_id     INTEGER PRIMARY KEY,
    total_reps     INTEGER NOT NULL DEFAULT 0,
    total_points   INTEGER NOT NULL DEFAULT 0,
    bad_move_count INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (session_id) REFERENCES sessions(session_id)
);
```

Appendix C: Linear Regression Pseudocode and Confidence Interval Derivation

The pseudocode below shows the entry-point function compute_performance_prediction(user_id, n_sessions). The implementation prefers scikit-learn's LinearRegression but falls back to numpy.polyfit if scikit-learn is unavailable.

```
def compute_performance_prediction(user_id, n_sessions):
    # 1. Fetch the most recent N session records for this user
    sessions = fetch_recent_sessions(user_id, n_sessions)
    if len(sessions) < 2:
        return PredictionResult.empty(reason='insufficient history')

    # 2. Compute engineered features per session
    speed   = [(s.reps / s.minutes) / 50.0       for s in sessions]
    volume  = [s.reps / 50.0                      for s in sessions]
    quality = [1.0 - (s.bad_moves / max(s.reps, 1)) for s in sessions]

    x = np.arange(len(sessions))

    # 3. Fit one linear regression per feature
    slope_s, intercept_s = fit_linreg(x, speed)
```

```
    slope_v, intercept_v = fit_linreg(x, volume)
    slope_q, intercept_q = fit_linreg(x, quality)

    # 4. Forecast next-session values
    x_next = len(sessions)
    pred_s = intercept_s + slope_s * x_next
    pred_v = intercept_v + slope_v * x_next
    pred_q = intercept_q + slope_q * x_next

    # 5. Confidence intervals (1.96 * SE)
    se_q   = sqrt(sum((quality[i] - (intercept_q + slope_q*i))**2
                      for i in range(len(sessions))) / max(len(sessions)-2,
1))
    ci_q   = (pred_q - 1.96 * se_q, pred_q + 1.96 * se_q)

    # 6. Composite scores
    endurance = 0.55 * pred_v + 0.45 * pred_s
    rating    = 0.45 * pred_s + 0.30 * pred_v + 0.25 * pred_q

    # 7. Training load scenarios
    scenarios = {
        'deload_80':     forecast_at_load(slope_v, intercept_v, x_next,
0.80),
        'maintain_100':  forecast_at_load(slope_v, intercept_v, x_next,
1.00),
        'overload_120': forecast_at_load(slope_v, intercept_v, x_next, 1.20),
    }

    # 8. Error metrics
    rmse, mae, r2 = compute_error_metrics(quality, slope_q, intercept_q)

    return PredictionResult(pred_s, pred_v, pred_q, ci_q,
                            endurance, rating, scenarios,
                            rmse, mae, r2, history=sessions)
```

Appendix D: TrainBot Prompt Template

The TrainBot system prompt is assembled from a fixed instruction preamble and a set of substitution slots populated from the database and the prediction engine output. The full template is shown below with substitution markers in angle brackets.

```
You are TrainBot, a personalised athletic coach for the NeuroMotion
system. Use the athlete profile and recent performance summary
supplied below to ground every recommendation in the athlete's
measured behaviour. Do not produce generic advice. Where the
athlete asks for a training report, respond with: (a) a brief
performance summary, (b) a four-week periodised workout plan,
(c) practical nutritional recommendations consistent with the
athlete's exercise profile.

ATHLETE PROFILE
---------------
Username:                <username>
Performance grade:       <performance_rating> / 1.00
Endurance score:         <endurance_score>    / 1.00
Predicted next-session
    speed (normalised):  <pred_speed>
    volume (normalised): <pred_volume>
    form quality:        <pred_quality> (95% CI <ci_low>..<ci_high>)

EXERCISE BREAKDOWN (last 30 days)
---------------------------------
Jumps:    <pct_jumps>%
Squats:   <pct_squats>%
Push-ups: <pct_pushups>%

LEADERBOARD POSITION
--------------------
Overall rank:  <overall_rank> / <overall_total>
Best tab:      <best_tab> (rank <best_rank>)

MOST COMMON FORM FAULTS
-----------------------
1. <fault_1>     (<count_1> occurrences)
2. <fault_2>     (<count_2> occurrences)
3. <fault_3>     (<count_3> occurrences)
```

```
FATIGUE SIGNAL
--------------
Fatigue score: <fatigue_score> / 100
Tier:          <fatigue_tier>
```